Análise e redução de volume de tráfego do FACEIT (CS2)

Captura global com Wireshark, classificação, filtragem e agregação em fluxos

[Nome do aluno 1] ([n.º]) [Nome do aluno 2] ([n.º])

[Unidade curricular] — [Curso / Instituição]

4 de novembro de 2026

Resumo

Capturou-se, com o Wireshark, todo o tráfego de uma máquina durante uma partida de CS2 na plataforma FACEIT (326 122 pacotes, 206,9 MB, cerca de 30 minutos). O serviço foi classificado em dois planos: o *plano de jogo* (um único fluxo UDP para `141.95.32.214:27031`, 62 % dos pacotes e 79 % dos bytes) e o *plano de controlo* (DNS, TLS e QUIC para `193.41.200.101/102`, cerca de 1 % do volume). Partindo da mesma captura global, quantificou-se a redução de volume obtida por filtragem (21,3 % dos bytes), truncagem de pacotes com *snaplen* 96 (87,5 %), *sampling* 1/10 combinado com *snaplen* (98,9 %) e agregação em fluxos com o Argus (99,5 % a 99,97 %).

1 Introdução e objetivos

Este trabalho consolida as etapas estudadas sobre captura e análise de tráfego. Os objetivos são:

- escolher um serviço (aqui, o FACEIT, com partidas de CS2), gerar tráfego e capturar o tráfego global com o Wireshark;
- estudar os critérios de classificação e filtragem mais eficazes (filtros de captura e de visualização);
- reduzir o volume de dados capturado/armazenado e quantificar a percentagem de otimização face à captura global, por filtragem, truncagem de pacotes, *sampling* e agregação;
- explorar uma ferramenta de agregação de pacotes em fluxos a partir de ficheiros `pcap`.

2 Metodologia

Captura. O Wireshark correu na própria máquina de jogo (`192.168.1.82`), em ligação por cabo, **sem filtro de captura e com pacotes inteiros** (captura global), durante 1 818,6 s (cerca de 30 minutos), com o FACEIT Anti-Cheat ativo. Antes da captura final fez-se um ensaio num servidor normal de CS2 (fora do FACEIT) para validar o procedimento.

Estratégia de análise. Fez-se uma única captura global, convertida para `pcap` simples (`editcap -F pcap`) para servir de *baseline*. Todos os cenários reduzidos foram **derivados offline desse mesmo ficheiro**, para que a comparação use exatamente o mesmo tráfego. Usaram-se o Wireshark/`tshark`, `editcap`, `capinfos`, `tcpdump` e o Argus (agregação em fluxos).

Métrica. A percentagem de otimização é

$$\text{redução} = \left(1 - \frac{\text{tamanho reduzido}}{\text{tamanho global}}\right) \times 100,$$

medida em bytes (tamanho do ficheiro) e em número de pacotes. O tamanho do ficheiro inclui 16 B de cabeçalho por pacote no formato `pcap`; o campo *Data size* do `capinfos` não reflete a truncagem, por isso não foi usado.

3 Caracterização do tráfego capturado

A captura global tem 326 122 pacotes, 201,6 MB de dados (212,6 MB em `pcapng`, 206,9 MB em `pcap`), com média de 618 B por pacote e 179 pacotes/s. A Tabela 1 mostra a composição por classe (as classes sobrepõem-se parcialmente, pelo que não se somam).

Tabela 1: Composição do tráfego global.

Classe	Pacotes	Bytes	% pacotes	% bytes
Servidor de jogo (`141.95.32.214`)	202 086	159 679 469	62,0	79,2
UDP `:19336` (`104.29.146.69`)	85 785	17 938 347	26,3	8,9
TCP (todo)	19 394	12 376 258	5,9	6,1
QUIC (todo)	7 226	9 622 529	2,2	4,8
mDNS / SSDP / DNS	4 576	1 307 245	1,4	0,6
Controlo FACEIT (`193.41.200.101/102`)	3 995	3 338 873	1,2	1,7

Plano de jogo. A conversa UDP `192.168.1.82:50088` ↔ `141.95.32.214:27031` domina a captura (Figura 1): começa aos 293,8 s, dura 1 520,5 s e tem 202 006 pacotes e 159,7 MB. O cliente envia 96 786 pacotes (70,7 MB, média de 730 B) e o servidor 105 220 (89,0 MB, média de 846 B). O débito é praticamente constante: cerca de 133 pacotes/s e 0,84 Mbit/s (aproximadamente 40 000 pacotes por cada 300 s). Os tamanhos concentram-se entre 640 e 1 279 B (69,3 % dos pacotes), com média de 790 B.

Plano de controlo. O DNS e o SNI (TLS e QUIC) identificam o serviço: `www.faceit.com`, `edge.faceit.com`, `chat.faceit.com`, `faceit.com`, `cdn-frontend.faceit-cdn.net`, `distribution.faceit-cdn.net` e `ac-client-ws.faceit.com`, todos resolvidos para `193.41.200.101` e `193.41.200.102` (Figuras 2 e 3), mais `analytics.faceitanalytics.com` (Cloudflare). O primeiro contacto ocorre aos 2,2 s (QUIC para `www.faceit.com`), há nova rajada de pedidos aos 225–245 s, antes de o fluxo de jogo começar, e um contacto a `ac-client-ws.faceit.com` aos 1 804,6 s. Este plano é pequeno: 3 995 pacotes e 3,3 MB.

Ruído. Os fluxos UDP para `104.29.146.69:19336` (um novo fluxo por minuto, 85 785 pacotes) são independentes do jogo: apareciam também na captura de ensaio. A sua origem não foi identificada. O restante é tráfego da rede local (mDNS, SSDP, *broadcasts*) e outras ligações HTTPS/QUIC.

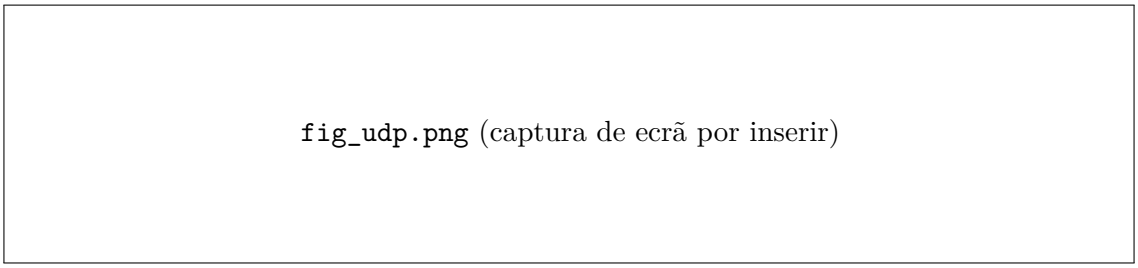


Figura 1: *Conversations* → UDP, ordenado por pacotes: o fluxo para `141.95.32.214:27031` domina.

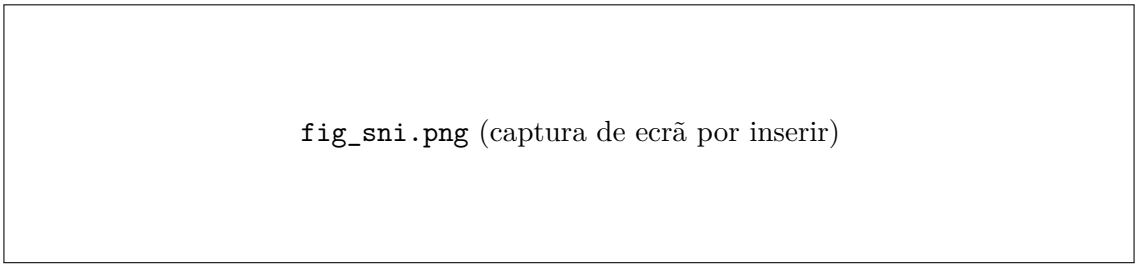


Figura 2: Filtro `tls.handshake.extensions_server_name contains "faceit"` (TLS e QUIC).

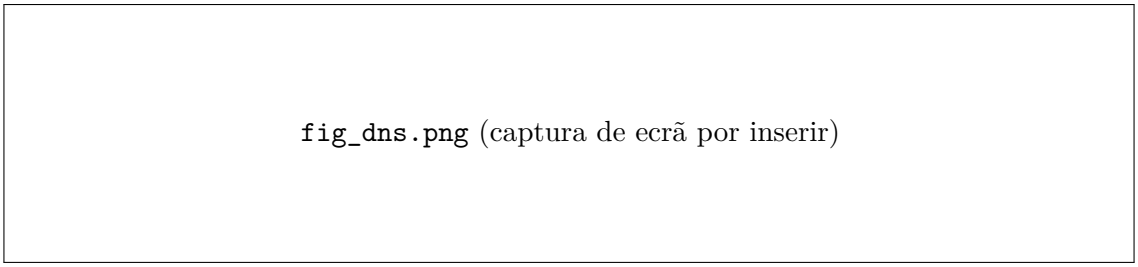


Figura 3: Filtro `dns.qry.name contains "faceit"`.

4 Critérios de classificação e filtros

A Tabela 2 resume os critérios estudados. O filtro de captura (BPF) só vê cabeçalhos (IP, protocolo, portas), pelo que critérios baseados em nomes (DNS/SNI) só são possíveis como filtros de visualização. A eficácia mede-se por cobertura (pacotes do jogo apanhados / pacotes do jogo) e precisão (pacotes do jogo apanhados / pacotes apanhados), tomando como referência a conversa do jogo (202 006 pacotes).

Tabela 2: Critérios de classificação e eficácia (BPF = filtro de captura; Vis. = filtro de visualização).

Critério	BPF	Vis.	Pacotes	Precisão
Servidor (IP)	`udp and host 141.95.32.214`	`ip.addr==141.95.32.214 && udp`	202 006	100 %
Qualquer protocolo com o IP	`host 141.95.32.214`	`ip.addr==141.95.32.214`	202 060	99,97 %
Gama de portas	`udp portrange 27000-27050`	`udp.port>=27000 && udp.port<=27050`	202 040	99,98 %
Só UDP	`udp`	`udp`	300 571	67,2 %
Controlo (IP)	`host 193.41.200.101 or host 193.41.200.102`	`ip.addr==193.41.200.101 || ip.addr==193.41.200.102`	3 995	—
Controlo (nome)	(não aplicável)	`dns.qry.name contains "faceit"; tls.handshake.extensions_server_name contains "faceit"`	—	—
Jogo + controlo	`host` servidor `or host` controlo (2 IPs)	—	206 055	—

Todos os filtros por IP ou por portas têm cobertura de 100 % do fluxo de jogo. A gama de portas apanha apenas 34 pacotes a mais do que o filtro por IP (precisão de 99,98 %) e tem a vantagem de não depender do IP do servidor, que muda de partida para partida. O filtro apenas por protocolo (`udp`) é demasiado largo: 33 % do que apanha não é jogo. Como critérios de deteção do fluxo de jogo funcionam bem a combinação de UDP, a gama de portas `27xxx` e a dominância por volume (*Conversations* ordenado por bytes), confirmada pelo DNS/SNI do plano de controlo.

5 Redução de volume

A Tabela 3 e a Figura 4 apresentam os cenários. Todas as percentagens são cumulativas face ao *baseline* (A) de 326 122 pacotes e 206 864 624 B.

Filtrar. Filtrar o fluxo de jogo reduz 21,25 % dos bytes mas 38,06 % dos pacotes: o jogo já é 79 % dos bytes capturados, enquanto o ruído é composto por muitos pacotes pequenos (os 85 785 pacotes de `:19336` pesam 26 % dos pacotes mas 9 % dos bytes). Incluir o plano de controlo no filtro custa pouco (19,60 % contra 21,25 %) e descreve o serviço de forma mais completa.

Não capturar o pacote inteiro. É a técnica com maior impacto individual. Os cabeçalhos Ethernet, IPv4 e UDP ocupam 42 B, por isso com *snaplen* 96 guardam-se os cabeçalhos e mais 54 B de payload. Com pacotes de 790 B em média, a redução total passa de 21,25 % para 87,50 % (84,38 % com 128 B e 90,63 % com 64 B). O custo é a perda do restante payload.

Tabela 3: Redução de volume face à captura global (percentagens em tamanho de ficheiro e em pacotes).

	Cenário	Pacotes	Bytes	Red. bytes	Red. pacotes
A	Captura global (*baseline*)	326 122	206 864 624	0 %	0 %
B1	Filtro `udp and host` servidor	202 006	162 905 309	21,25 %	38,06 %
B3	Filtro gama de portas 27000–27050	202 040	162 951 481	21,23 %	38,05 %
B4	Filtro só `udp`	300 571	193 613 964	6,41 %	7,83 %
B5	Filtro jogo + controlo FACEIT	206 055	166 312 206	19,60 %	36,82 %
C1	B1 + *snaplen* 128	202 006	32 311 732	84,38 %	38,06 %
C2	B1 + *snaplen* 96	202 006	25 856 584	87,50 %	38,06 %
C3	B1 + *snaplen* 64	202 006	19 392 644	90,63 %	38,06 %
C4	B5 + *snaplen* 96	206 055	26 333 772	87,27 %	36,82 %
D1	B1 + *sampling* 1/10	20 200	16 376 628	92,08 %	93,81 %
D2	B1 + *snaplen* 96 + *sampling* 1/10	20 200	2 262 424	98,91 %	93,81 %
E1	Fluxos Argus, global (registos de 5 s)	—	1 004 872	99,51 %	—
E2	Fluxos Argus, global (1 registo/fluxo, CSV)	—	64 839	99,97 %	—
E3	Fluxos Argus, só servidor (CSV, 1 fluxo)	—	229	≈100 %	—

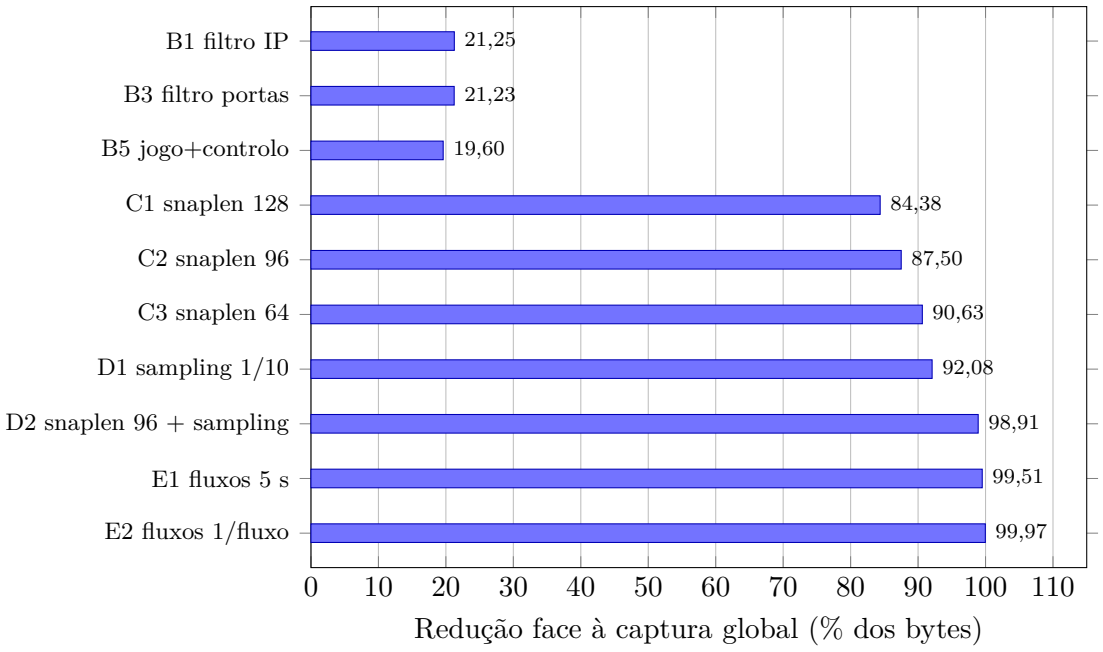


Figura 4: Redução de bytes por cenário.

Sampling. Guardar 1 pacote em 10 (por ordem de pacote) dá 92,08 % sem truncagem e 98,91 % com *snaplen* 96 (2,26 MB, dos quais cerca de 0,32 MB são cabeçalhos de pacote do formato `pcap`). As estatísticas de volume mantêm-se: extrapolando $\times 10$, o erro é $-0{,}003$ % nos pacotes e $+0{,}54$ % nos bytes. Perdem-se as sequências de pacotes consecutivos.

6 Agregação de pacotes em fluxos

Usou-se o Argus (`argus -r f.pcap -w f.argus`), que emite registos de estado de cerca de 5 s por fluxo, e o `racluster` para obter um registo por fluxo (5-tuplo), exportado para CSV com o `ra`. Do *baseline* de 206,9 MB obtêm-se 1,0 MB em registos de 5 s (99,51 %), 122,7 kB em binário com um registo por fluxo (99,94 %) e 64,8 kB em CSV (99,97 %), com 797 fluxos (mais um registo de gestão do Argus). Aplicado ao filtro do servidor, o fluxo de jogo fica reduzido a um único registo de 229 B no CSV.

Os totais do fluxo de jogo no Argus (70 661 525 B do cliente e 89 011 664 B do servidor) somam 159 673 189 B, exatamente o volume de dados do pcap filtrado. A agregação conserva o 5-tuplo, o instante de início, a duração e os pacotes e bytes por sentido, mas perde a temporização e o tamanho de cada pacote e o payload. Os registos de 5 s (E1) conservam ainda a evolução temporal do fluxo.

7 Conclusões

- O tráfego do FACEIT divide-se em plano de jogo (UDP `141.95.32.214:27031`, 79 % dos bytes) e plano de controlo (DNS/TLS/QUIC para `193.41.200.101/102`, cerca de 1 %).
- Os critérios mais eficazes para detetar o fluxo de jogo são UDP, gama de portas `27000-27050` e dominância por volume; o DNS/SNI confirma o serviço. O filtro apenas por protocolo é demasiado largo.
- Filtrar poupa pouco em bytes (21 %) porque o jogo domina a captura. As maiores poupanças vêm do *snaplen* (87,5 %), do *sampling* (98,9 % em conjunto) e da agregação em fluxos (99,5 % a 99,97 %).
- A escolha depende da utilização: cabeçalhos por pacote $\Rightarrow$ filtro e *snaplen* 96; estatísticas de volume $\Rightarrow$ *sampling*; contabilização e retenção prolongada $\Rightarrow$ fluxos.

Limitações. Analisou-se uma única captura de uma única partida; o IP e a porta do servidor mudam entre partidas. Os cenários foram derivados offline da captura global (não capturados ao vivo). O *sampling* é determinístico (1 em 10 por ordem de pacote). As percentagens em ficheiro incluem o cabeçalho de 16 B por pacote do formato `pcap`. Só se usou o Argus para a agregação em fluxos.

A Comandos utilizados

```
# Baseline
capinfos faceit.pcapng
editcap -F pcap faceit.pcapng A_global.pcap

# Caracterizacao
tshark -r A_global.pcap -q -z io,phs
tshark -r A_global.pcap -q -z conv,udp
```

```
tshark -r A_global.pcap -q -z plen,tree,"ip.addr==141.95.32.214"
tshark -r A_global.pcap -Y 'dns.flags.response==1 && dns.qry.name contains "faceit"' -T fields -
    e frame.time_relative -e dns.qry.name -e dns.a
tshark -r A_global.pcap -Y 'tls.handshake.extensions_server_name contains "faceit"' -T fields -e
     frame.time_relative -e ip.dst -e tls.handshake.extensions_server_name

# Filtros (BPF, aplicados offline)
tcpdump -r A_global.pcap -w B1_udp_host.pcap  "udp and host 141.95.32.214"
tcpdump -r A_global.pcap -w B3_portrange.pcap "udp portrange 27000-27050"
tcpdump -r A_global.pcap -w B4_udp.pcap       "udp"
tcpdump -r A_global.pcap -w B5_game_ctrl.pcap "host 141.95.32.214 or host 193.41.200.101 or host
     193.41.200.102"

# Snaplen e sampling
editcap -s 96 B1_udp_host.pcap C2_snaplen96.pcap
tshark -r C2_snaplen96.pcap -Y "frame.number % 10 == 0" -F pcap -w D2_snaplen_sampling.pcap

# Fluxos (Argus)
argus -r A_global.pcap -w E1_global.argus
racluster -r E1_global.argus -m proto saddr sport daddr dport -w E2_global_cluster.argus
ra -r E2_global_cluster.argus -n -c , -s stime dur proto saddr sport daddr dport spkts dpkts
    sbytes dbytes > E2_global_flows.csv

# Medicao
capinfos -T -m -M -c -s *.pcap
```